

COLLEGE OF BUSINESS EDUCATION



DEPARTMENT: MATHEMATICS AND INFORMATION TECHNOLOGY

PROGRAMME: BACHELOR OF INFORMATION TECHNOLOGY (BIT 2)

COURSE: INTERNET AND WEB DEVELOPMENT

ASSIGNMENT TYPE: INDIVIDUAL ASSIGNMENT

SYSTEM DOCUMENTATION

FUSSY EXPRESS ONLINE BUS TICKET BOOKING SYSTEM

PREPARED BY:FRANCE GAUFRID FUSSY

REGISTRATION NUMBER: 03.3154.01.01.2024

ISSUE DATE: 16/06/2026

PRESENTATION DATE: 16/07/2026

LIVE URL: <https://fussy0064.duckdns.org/>

PUBLIC IP: 16.170.204.159

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Public bus transport is one of the most heavily used means of intercity travel in Tanzania, including routes connecting Dar es Salaam to other regions. Historically, tickets for these journeys have been sold manually at bus company counters, which creates several recurring problems: Passengers must travel to a physical counter to check seat availability and prices, seat allocation is done on paper or in disconnected spreadsheets, and bus operators have no centralized, real-time way to track bookings, revenue, or fleet utilisation across multiple routes and schedules.

This project, Fussy Express, is a web-based Online Bus Ticket Booking System developed to address these problems. It allows passengers to search available trips by route and travel date, view a live seat map, and book a specific seat online. It allows an Admin to manage the bus fleet, routes and schedules, review and confirm incoming bookings, and generate operational reports. A Super Admin role sits above Admin to manage staff/user accounts system-wide, including secure password resets.

1.2 Problem Statement

Manual, counter-based bus ticket booking is slow, error-prone, and offers no live visibility of seat availability to a passenger before they physically visit a counter. It also gives bus operators no reliable digital record of bookings, payments, or revenue by route, making planning and accountability difficult. There is a need for a secure, database-driven web application that lets passengers book seats remotely in real time, while giving administrators the tools to manage the fleet and review/confirm bookings before they are finalised.

1.3 Objectives

General Objective

To design and develop a secure, object-oriented, database-driven web application that automates the process of searching, booking, confirming, and managing intercity bus tickets.

Specific Objectives

- To design a normalized (3NF) relational database that stores users, buses, routes, schedules and bookings without redundancy.
- To implement the system in PHP using Object-Oriented Programming principles encapsulation, inheritance, polymorphism and abstraction without relying on any PHP framework.
- To implement secure user authentication, role-based access control (Passenger, Admin, Super Admin), and session management.
- To implement row-level encryption of sensitive personal data before it is stored in the database, with controlled decryption on retrieval.
- To implement an admin confirmation workflow so that a booking only becomes final after review, with automatic email notification to both the passenger and the administrators.
- To provide reporting features (tickets sold, revenue by route, bookings over time) for management decision-making.
- To deploy and host the completed system on a live AWS EC2 instance, accessible via a public URL.

1.4 Significance of the Study

The system demonstrates practical, real-world application of OOP and secure web-development principles to a problem that affects everyday commuters and bus operators.

For me, it demonstrates competence in PHP/MySQL development without frameworks, database normalization, encryption, and cloud deployment all core outcomes of the Internet and Web Development course.

1.5 Scope and Limitations

The system covers passenger self-service booking, seat-map selection, admin fleet/route/schedule management, an admin booking-confirmation workflow, and Super Admin user management. It does not integrate a live third-party payment gateway (e.g., mobile money or card processing) the amount due is calculated and recorded, and payment is currently assumed to be settled through the operator's existing channels.

CHAPTER TWO: REQUIREMENTS ANALYSIS

2.1 Functional Requirements

The system shall provide the following capabilities, grouped by user role:

2.1.1 Passenger

- Register a new account and log in securely.
- Search for available trips by departure city, destination city, and travel date.
- View a visual seat map for a chosen schedule, with already-booked seats disabled.
- Book a specific seat, optionally on behalf of another named passenger.
- Receive an email when a booking is submitted (pending) and a second email once it is confirmed by an Admin.
- View and cancel their own bookings (pending or confirmed).
- View and print a generated ticket.

2.1.2 Admin

- Perform full CRUD (Create, Read, Update, Delete) operations on Buses, Routes, and Schedules.
- Review all bookings pending, confirmed, and cancelled in one place, filterable by status.
- Confirm a pending booking (triggering an automatic confirmation email to the passenger) or cancel it.
- View a dashboard with system-wide statistics: tickets sold, pending bookings, total revenue, fleet size.
- Generate reports: total tickets sold, total revenue, revenue by route, bookings in the last 30 days.

2.1.3 Super Admin

- Everything an Admin can do (Super Admin inherits the Admin role).
- List all user accounts with decrypted personal details, for management purposes only.
- Create, edit, or delete user accounts of any role.
- Reset any user's password to a system-generated default value, emailed automatically to that user.

2.2 Non-Functional Requirements

- Security: passwords must never be stored or transmitted in plain text; sensitive personal data must be encrypted at rest.
- Usability: the interface must be responsive and usable on both desktop and mobile screens.
- Reliability: seat booking must never allow two passengers to be assigned the same seat on the same schedule.
- Availability: the system must be reachable over the public internet via HTTPS, hosted on AWS.
- Maintainability: code must be organized using OOP classes with a single, clear responsibility each, and must not depend on a PHP framework.

2.3 Tools and Technologies Used

Category	Tool / Technology
Server-side language	PHP (procedural as an entry points + Object-Oriented classes)
Database	MySQL
Database access	PDO (PHP Data Objects)
Web server	Apache 2
Front-end	HTML, CSS , vanilla JavaScript
Encryption	OpenSSL - AES-256-CBC
Password storage	PHP password_hash()
Version control	Git and GitHub
Hosting / Deployment	Amazon Web Services (AWS) EC2, Ubuntu Server 24.04 LTS
Domain / HTTPS	DuckDNS dynamic domain + Let's Encrypt (Certbot) TLS certificate

No PHP framework (e.g., Laravel, CodeIgniter, Symfony, Yii, CakePHP) was used at any point; all routing, database access, and business logic were written directly in PHP using the classes described in Chapter Three.

CHAPTER THREE: SYSTEM DESIGN

3.1 System Architecture Overview

The system follows a simple layered architecture written in plain PHP:

- Presentation layer => PHP scripts under public/ (e.g., search.php, book.php, ticket.php, admin/*.php, superadmin/*.php) that render HTML and handle form submissions.
- Domain / business logic layer => PHP classes under classes/ (User, Passenger, Admin, SuperAdmin, Booking, Bus, Route, Schedule, Ticket, Mailer) that encapsulate all rules and validation.
- Data-access layer => an abstract DatabaseModel base class wrapping PDO, used by every persisted class for connection handling and prepared-statement execution.
- Security layer => security/EncryptionHelper.php (AES-256-CBC field encryption) and password hashing used throughout the domain layer.

This separation means a request such as booking a seat flows: public/book.php (presentation) → Passenger::bookTicket() (domain logic) → Booking::save() (data access, inside a database transaction) → EncryptionHelper (encrypts seat number, passenger name, contact details before they are written to MySQL).

3.2 Database Design (Third Normal Form)

The database bus_booking_system consists of five core tables. The design deliberately keeps each fact in exactly one place e.g., a route's price is stored once in routes and never copied into schedules or bookings so the schema satisfies 1NF (atomic columns, no repeating groups), 2NF (every table uses a single column surrogate primary key, so partial dependency on part of a composite key is impossible), and 3NF (no non-key column depends on another non-key column; all such data is reached through a foreign key relationship instead).

3.2.1 Entity Relationship Summary

- users (1) - (many) bookings; a user can make many bookings.
- buses (1) -(many) schedules; a bus can be assigned to many scheduled trips (at different times).
- routes (1) -(many) schedules; a route can be run by many scheduled trips.
- schedules (1) - (many) bookings; a schedule (a specific bus + route + departure time) can have many seat bookings, up to its seat capacity.

3.2.2 Table Structures

Sensitive personal-data columns (marked ENC) are stored AES-256-CBC encrypted, never in plain text. Columns marked **HASH** store a one-way SHA-256 hash of the normalized plain value purely so the application can search/check-uniqueness on encrypted fields without decrypting every row.

users

Column	Type	Notes
id	INT, PK, AUTO_INCREMENT	Surrogate primary key
name_enc	TEXT *ENC*	Encrypted full name
email_enc / email_hash	TEXT *ENC* / CHAR(64) *HASH*	Encrypted email + lookup hash (UNIQUE)
phone_enc	TEXT *ENC*, nullable	Encrypted phone number
password_hash	VARCHAR(255)	bcrypt hash never encrypted/decrypted, only verified
role	ENUM(admin, passenger, super_admin)	Distinguishes account type
created_at / updated_at	TIMESTAMP	Audit timestamps

buses

Column	Type	Notes
id	INT, PK	Surrogate primary key
bus_number_enc	TEXT *ENC*	Encrypted fleet/registration number
bus_type	ENUM(Standard, Luxury)	Determines seat-map layout
total_seats	SMALLINT UNSIGNED	Physical seating capacity

routes

Column	Type	Notes
id	INT, PK	Surrogate primary key
departure_city_enc / _hash	TEXT *ENC* / CHAR(64) *HASH*	Encrypted city + searchable hash
destination_city_enc / _hash	TEXT *ENC* / CHAR(64) *HASH*	Encrypted city + searchable hash
distance_km	INT UNSIGNED, nullable	Informational
price	DECIMAL(10,2)	Base fare kept in plain numeric form for SUM()/reporting

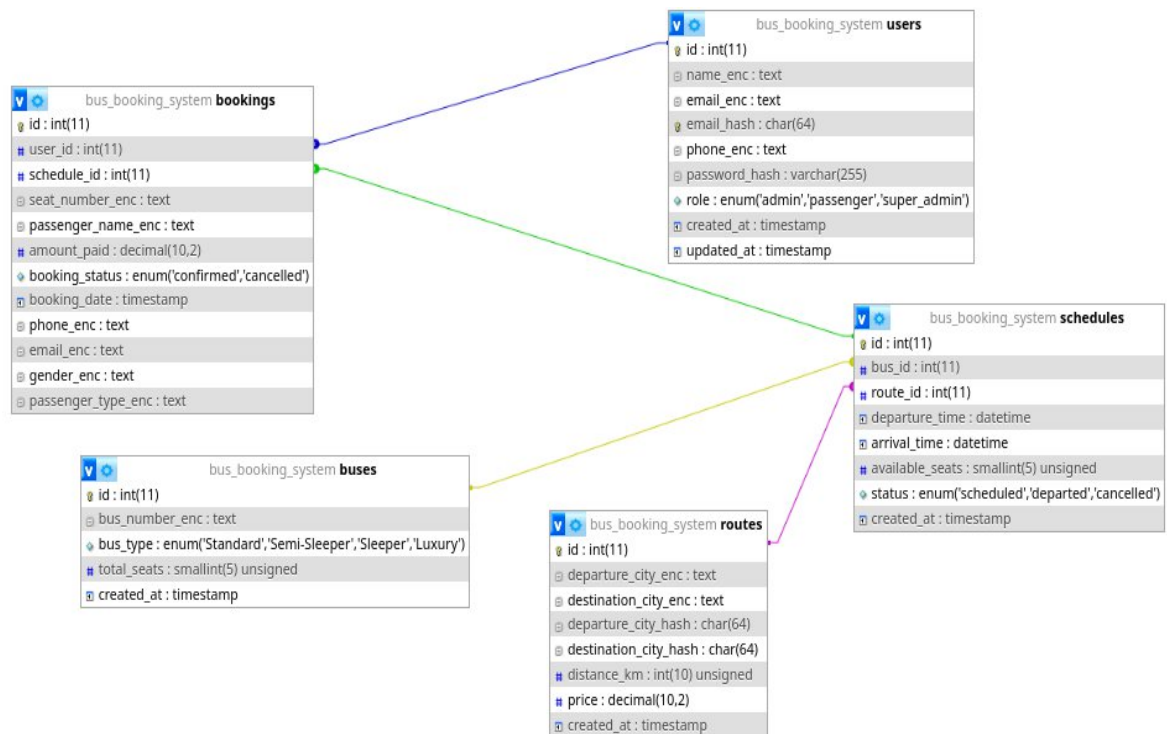
schedules

Column	Type	Notes
id	INT, PK	Surrogate primary key
bus_id, route_id	INT, FK	Links a bus + route for one specific trip
departure_time / arrival_time	DATETIME	Trip timing
available_seats	SMALLINT UNSIGNED	Live running counter, decremented/incremented on booking/cancel
status	ENUM(scheduled, departed, cancelled)	Operational state

bookings

Column	Type	Notes
id	INT, PK	Surrogate primary key
user_id, schedule_id	INT, FK	Who booked, and on which trip
seat_number_enc / passenger_name_enc	TEXT *ENC*	Encrypted seat label and passenger name
phone_enc / email_enc / gender_enc / passenger_type_enc	TEXT *ENC*, nullable	Optional encrypted contact/passenger details
amount_paid	DECIMAL(10,2)	Kept plain for reporting/SUM() aggregation
booking_status	ENUM(pending, confirmed, cancelled)	Starts pending; Admin confirms or cancels
booking_date	TIMESTAMP	When the booking was made

DATABASE DESIGN



3.3 Object-Oriented Design

The domain layer is built around a small class hierarchy that deliberately demonstrates all four OOP pillars required by the assignment brief.

3.3.1 Class Hierarchy

```
fussy@fussy-0064:/opt/lampp/htdocs/bus-booking-system$ tree -d
.
├── classes
├── config
├── database
├── docs
├── public
│   ├── admin
│   ├── css
│   ├── partials
│   └── superadmin
└── security

11 directories
```

3.3.2 Abstraction

DatabaseModel is declared abstract and is never instantiated directly; it exists purely to give every persisted entity a shared, reusable way to obtain a PDO connection and run parameterised queries (query()/execute()), hiding the raw SQL plumbing from the classes that use it.

```
abstract class DatabaseModel
{
    protected static ?PDO $connection = null;

    protected static function getConnection(): PDO { /* ... */ }
    protected static function query(string $sql, array $params = []): array { /* ... */ }
    protected static function execute(string $sql, array $params = []): int { /* ... */ }
}
```

3.3.3 Encapsulation

Every entity class keeps its properties private and only exposes controlled access through getters and setters, so internal representation (e.g., that a seat number is stored encrypted) never leaks to calling code.

```
class Booking extends DatabaseModel
{
    private ?int $id = null;
    private string $seatNumber;
    private string $bookingStatus = 'pending';

    public function getSeatNumber(): string { return $this->seatNumber; }
    public function setSeatNumber(string $seatNumber): void { $this->seatNumber = $seatNumber; }
}
```

3.3.4 Inheritance

User implements everything common to any account registration, login, password verification, session-relevant getters. Passenger, Admin, and SuperAdmin extend it (SuperAdmin extends Admin, so a Super Admin automatically inherits every Admin capability as well as User's), reusing that logic instead of duplicating it.

```
class Passenger extends User { /* adds bookTicket(), getMyBookings() */ }
class Admin extends User { /* adds Bus/Route/Schedule CRUD, booking review */ }
class SuperAdmin extends Admin { /* adds user management, password resets */ }
```

3.3.5 Polymorphism

User declares displayDashboard(); each subclass overrides it to return role-appropriate data through the exact same method call, so calling code (e.g., the dashboard page) can call \$user->displayDashboard() without knowing or caring which subclass it actually received.

```
// Passenger
public function displayDashboard(): array {
    return ['title' => 'My Dashboard', 'bookings' => Booking::findByUser($this->getId())];
}

// Admin
public function displayDashboard(): array {
    return ['title' => 'Admin Dashboard',
        'totalTickets' => Booking::countConfirmedTickets(),
        'pendingBookings' => Booking::countPending(),
        'totalRevenue' => Booking::totalRevenue()];
}
```

3.4 Security Design

3.4.1 Row-Level Encryption of Sensitive Data

All personally identifiable data user names, emails, phone numbers, bus registration numbers, route cities, passenger names, seat numbers, gender, and passenger type is encrypted at the application layer, in security/EncryptionHelper.php, before it is ever written to the database, using AES-256-CBC via PHP's OpenSSL extension.

- Key management: the 256-bit key is derived (via SHA-256) from a secret value defined once in config/env.local.php (which is excluded from version control by .gitignore) and never hard-coded in source. In production this can be moved to AWS Secrets Manager / SSM Parameter Store instead of a flat file.
- Random IV per value: a fresh, random Initialization Vector (IV) is generated for every single encryption call, so encrypting the same plain text twice never produces the same cipher text this defeats pattern-matching/frequency analysis on the encrypted column.
- Storage format: the IV is not secret; it is prepended to the cipher text and the pair is base64-encoded before being stored as one TEXT value, so it is available again at decryption time.

- Searchable hashing: because encrypted values cannot be searched with a plain SQL WHERE clause, a separate, deterministic SHA-256 hash of the normalized (lower-cased, trimmed) plain text is stored alongside (e.g., email_hash, departure_city_hash). The application looks up rows by hashing the incoming search term the same way and matching the hash never by decrypting every row.
- Decryption: only performed on demand, inside the domain classes, immediately before displaying authorized data to an Admin/Super Admin or the owning Passenger never exposed directly to the database layer or logs.

3.4.2 Password Security

Passwords are never encrypted (which would be reversible) they are hashed with bcrypt via PHP's password_hash()/password_verify(), which is one-way and includes an automatically-generated per-password salt. Even a Super Admin resetting another user's password cannot see the old password; a reset simply generates and hashes a brand-new one.

3.4.3 Authentication, Sessions and Access Control

- Session cookies are hardened: http only (unreachable from JavaScript), samesite=Lax, and secure is automatically enabled once the app detects it is being served over HTTPS.
- session_regenerate_id(true) is called on every successful login to prevent session-fixation attacks.
- Every state-changing form submission carries a CSRF token, verified server-side with hash_equals() before any action is performed.
- Role-based access control: dedicated requirePassenger()/requireAdmin()/requireSuperAdmin() guard functions run at the top of every protected page, redirecting unauthenticated or under-privileged visitors away before any business logic executes.
- All database queries use PDO prepared statements with bound parameters, eliminating SQL-injection as an attack vector.
- All dynamic output is passed through an e()/htmlspecialchars() helper before being written into HTML, preventing stored/reflected Cross-Site Scripting (XSS).

CHAPTER FOUR: IMPLEMENTATION

4.1 Key Features Implemented

4.1.1 Passenger-Facing Features

- Registration and login with bcrypt-hashed passwords and encrypted personal data.
- Search for trips by departure city, destination city, and date.
- Interactive seat-map booking screen: Luxury buses render a premium 2+1 layout; Standard/Semi-Sleeper/Sleeper buses render a 2+2 layout with aisle, door and toilet markers. Seats already reserved (pending or confirmed) are shown disabled.
- Booking submission creates a pending booking immediately, reserving the seat, and emails the passenger a "booking received" notice.
- My Bookings page listing the passenger's own bookings with a status badge (Pending / Confirmed / Cancelled) and a Cancel option.
- Printable ticket view once a booking is confirmed.

4.1.2 Admin-Facing Features

- Full CRUD management of Buses, Routes, and Schedules.
- A unified Bookings page listing every booking, filterable by All / Pending / Confirmed / Cancelled, so nothing has to be tracked outside the system.
- One-click Confirm on a pending booking this automatically emails the passenger a confirmation notice.
- Automatic email alert to every Admin/Super Admin account the moment a new booking is placed, containing a direct link to that booking's confirm/cancel screen.
- Guard rails that block deleting a Bus, Route, or Schedule that still has booking history attached, to protect revenue/audit records from being silently destroyed by a cascading delete.
- Dashboard summarizing tickets sold, pending bookings, total revenue, and fleet size.
- Reports: total tickets sold, total revenue, revenue by route, and bookings in the last 30 days.

4.1.3 Super Admin-Facing Features

- User management: list every account (with decrypted name/email/phone for authorized administrative viewing), edit, or delete any account.
- Password reset that auto-generates a new default password from the first letter of the user's name plus their registration date/time, and emails it to them automatically the admin never sees or types the user's actual password.
- Protection against removing the very last Super Admin account or deleting one's own account, preventing the system from being left with no administrator.

4.2 Screenshots of Major Interfaces

The screenshots below illustrate the completed system. Each placeholder box should be replaced with an actual screen capture of the corresponding page taken from the live deployment before final submission.

4.2.1 Passenger Interfaces

Fussy Express

DEPARTURES

Dar es Salaam to Tanga

DEPARTURE CITY

Dar es Salaam

DESTINATION CITY

Tanga

TRAVEL DATE

mm / dd / yyyy

Search

Dar es Salaam to Tanga

DEPARTS	BUS	SEATS LEFT
16 Jul, 16:45	FAZ-278 - Luxuzy	62 / 63

Tsh 35,000.00

PER SEAT

Select seat

Home / trip search page

LUXURY

A3

A4

B3

B4

C3

C4

D3

D4

E3

E4

TOILET

DOOR

G3

G4

H3

H4

I3

I4

J3

J4

K3

K4

A2

A1

B2

B1

C2

C1

D2

D1

E2

E1

F2

F1

G2

G1

H2

H1

I2

I1

J2

J1

K2

K1

Dar es Salaam to Tanga

DEPARTS	BUS	SEATS LEFT
16 Jul, 16:45	FAZ-278 - Luxuzy	62

Tsh 35,000.00

PER SEAT

PASSENGER NAME

Fussy

PHONE NUMBER

0759050666

EMAIL ADDRESS

fussy0906@proton.me

GENDER

Male

PASSENGER TYPE

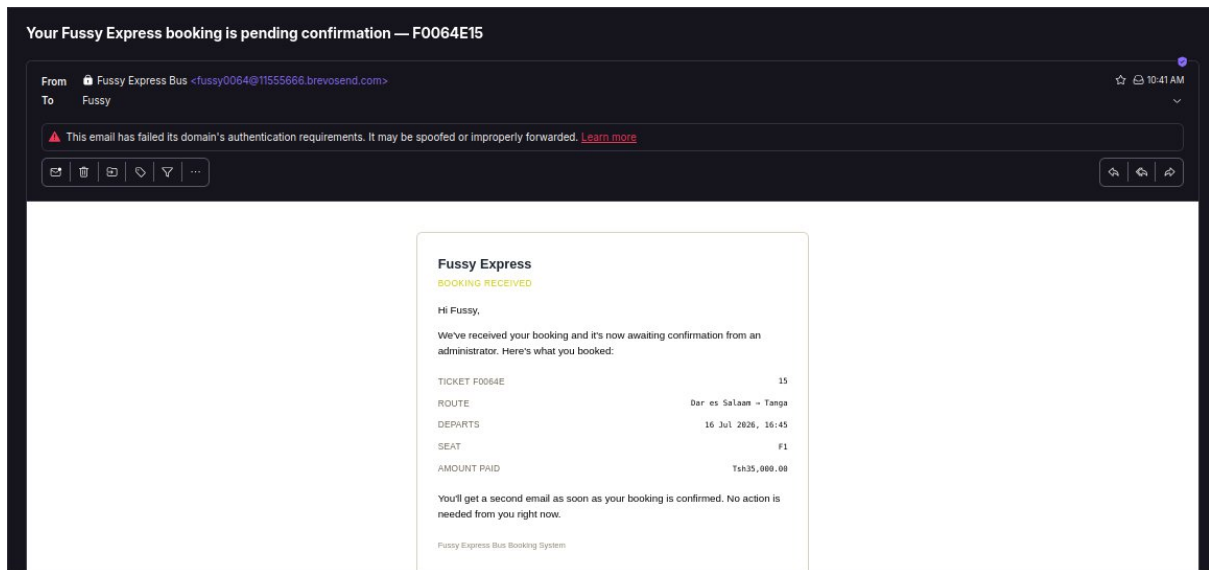
☒ ADULT
☐ STUDENT
☐ CHILD

SEAT NUMBER

F1

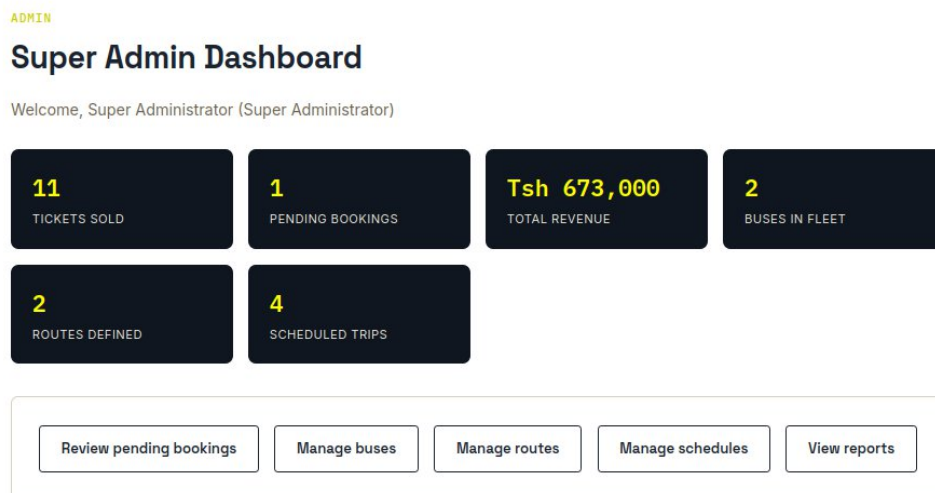
Confirm booking - Tsh 35,000.00

Seat-map selection screen



Booking confirmation / pending notice email

4.2.2 Admin Interfaces



Admin dashboard

Bookings

Confirming a pending booking automatically emails the passenger. So does cancelling.

All Pending Confirmed Cancelled

Booking #14 cancelled.

TICKET #	PASSENGER	CONTACT	SEAT	AMOUNT	STATUS	BOOKED ON	
#15	Fussy	fussy0906@proton.me	F1	Tsh 35,000.00	PENDING	16 Jul 2026, 07:41	<button>View</button> <button>Confirm</button> <button>Cancel</button>
#14	Princess Bosco	gaudenciaoscar2005@gmail.com	A4	Tsh 35,000.00	CANCELLED	15 Jul 2026, 21:58	<button>View</button>

Admin Bookings page (All / Pending / Confirmed / Cancelled)

Buses

Add a new bus

BUS NUMBER

e.g. RL-204

BUS TYPE

Standard

TOTAL SEATS

54

Add bus

ID	BUS NUMBER	TYPE	TOTAL SEATS	
#2	EZK	Standard	54	<button>Edit</button> <button>Delete</button>

Admin Buses management

Add a new route

DEPARTURE CITY

-- Select Departure City/Region --

DESTINATION CITY

-- Select Destination City/Region --

DISTANCE (KM, OPTIONAL) PRICE (TSH)

Add route

ID	DEPARTURE	DESTINATION	DISTANCE	PRICE		
#2	Dar es Salaam	Tanga	98 km	Tsh 35,000.00	Edit	Delete
#1	Dar es Salaam	Songea	580 km	Tsh 71,000.00	Edit	Delete

Admin Routes management

Schedules

Create a new schedule

BUS

Select a bus

ROUTE

Select a route

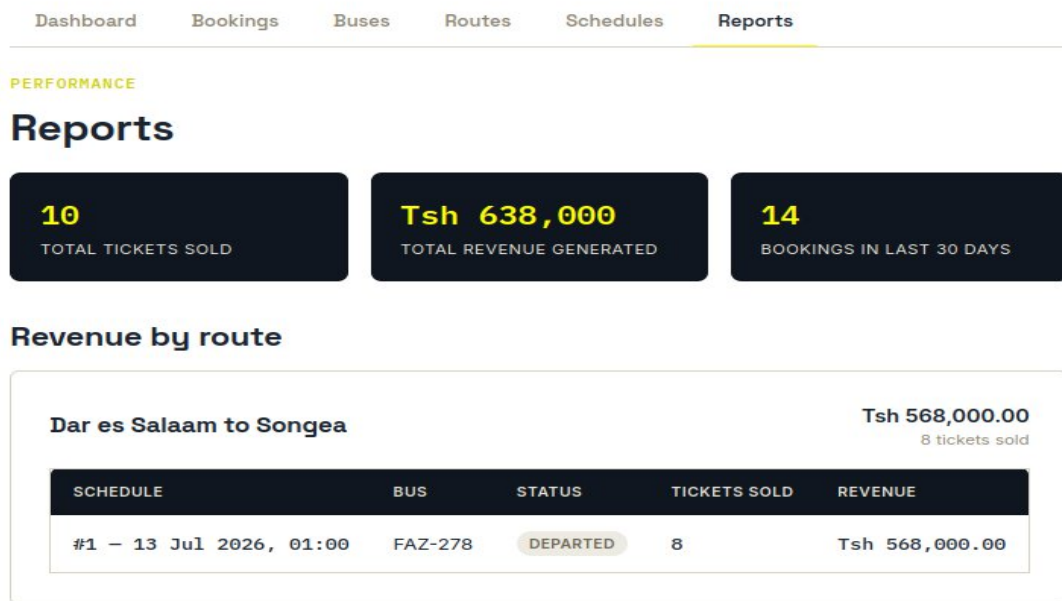
DEPARTURE TIME ARRIVAL TIME (OPTIONAL)

mm/dd/yyyy, --:- mm/dd/yyyy, --:-

Create schedule

ID	ROUTE	BUS	DEPARTURE	SEATS	STATUS
----	-------	-----	-----------	-------	--------

Admin Schedules management

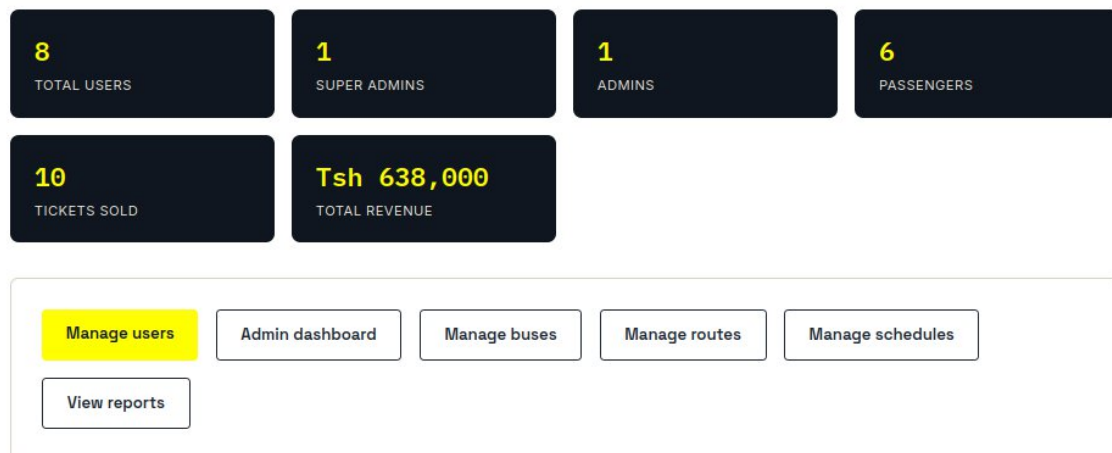


Admin Reports page

4.2.3 Super Admin Interfaces

Super Admin Dashboard

Welcome, Super Administrator (Super Administrator)



Super Admin dashboard

Fussy Express

PASSWORD RESET REQUESTED

Hi Fussy,

Use the button below to choose a new password for your account. This link can only be used once, and expires in 60 minutes.

Set a new password

If the button doesn't work, copy and paste this link into your browser:

<https://fussy0064.duckdns.org/reset-password.php?token=271f7caa7ec6089b4ff51a58d05fbb163a90625e146789f8a1f7c6862f39e347>

If you didn't request this, you can safely ignore this email — your password won't change unless the link above is used.

Fussy Express Bus Booking System

Password reset confirmation and resulting email

CHAPTER FIVE: HOSTING AND DEPLOYMENT (AWS EC2)

The completed system was deployed on a live Amazon Web Services (AWS) EC2 instance, running Ubuntu Server with Apache, PHP, and MySQL, and is reachable publicly over HTTPS at <https://fussy0064.duckdns.org/>. The steps below were followed to provision and configure the server.

5.1 Launching the EC2 Instance

1. Signed in to the AWS Management Console and opened the EC2 service.
2. Chose Launch Instance and selected the Ubuntu Server 24.04 LTS Amazon Machine Image (AMI).
3. Selected the t2.micro / t3.micro instance type (eligible for the AWS Free Tier and sufficient for this application).
4. Created and downloaded a new key pair (.pem file), used to authenticate SSH access to the instance.
5. Configured the Security Group to allow inbound traffic on: SSH (port 22, restricted to a known IP where possible), HTTP (port 80), and HTTPS (port 443).
6. Launched the instance and recorded its public IPv4 address.

AWS EC2 console instance running

5.2 Connecting to the Instance

Giving Permission pem-key to be Used by ssh -i

```
fussy@fussy-0064:~$ chmod 400 ~/Downloads/fussy.pem
```

Connecting to the Server

```
fussy@fussy-0064:~$ ssh -i ~/Downloads/fussy.pem ubuntu@16.170.204.159
```

5.3 Installing the Web Server Stack

Apache, PHP (with the MySQL, extensions), and MySQL Server were installed directly on the instance:

```
ubuntu@ip-172-31-43-195:~$ sudo apt update && sudo apt upgrade -y
sudo apt install -y apache2 mysql-server \
    php php-mysql php-mbstring php-xml php-curl libapache2-mod-php
sudo systemctl enable apache2 mysql
sudo systemctl start apache2 mysql
sudo mysql_secure_installation
```

Terminal stack installation / systemctl status output

5.4 Creating the Application Database

```
sudo mysql -u root -p
CREATE USER 'busapp'@'localhost' IDENTIFIED BY '<strong-password>';
GRANT ALL PRIVILEGES ON bus_booking_system.* TO 'busapp'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

5.5 Deploying the Application Code

```
cd /var/www
sudo git clone https://github.com/fussy0064/TicketBooking-System.git
sudo chown -R $USER:$USER /var/www/TicketBooking-System
cd TicketBooking-System
mysql -u busapp -p bus_booking_system < database/schema.sql
```

5.6 Application Configuration and Secrets

A config/env.local.php file (excluded from version control) was created from the provided template and populated with the live database credentials, a freshly-generated 256-bit AES encryption key, and outbound-email (SMTP) settings, with APP_ENV set to production and APP_DEBUG set to false so raw errors are never shown to visitors.

```
cp config/env.example.php config/env.local.php
```

Creating Encryption Key

```
php -r "echo bin2hex(random_bytes(32));"
```

```
Configuring .env
nano config/env.local.php
```

5.7 Web Server (Virtual Host) Configuration

Apache's document root was pointed directly at the application's public/ folder rather than the project root, so that the classes/, config/, security/ and database/ folders which hold credentials, the encryption key, and raw SQL sit outside the web server's reach entirely:

```
<VirtualHost *:80>
    ServerName fussy0064.duckdns.org
    DocumentRoot /var/www/TicketBooking-System/public

    <Directory /var/www/TicketBooking-System/public>
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

```
sudo a2ensite ticketbooking.conf
sudo a2dissite 000-default.conf
```

```
sudo a2enmod rewrite
sudo systemctl restart apache2
```

5.8 Domain Name and HTTPS (TLS) Certificate

A free dynamic subdomain, `fussy0064.duckdns.org`, was registered through DuckDNS and pointed at the EC2 instance's public IP address. A free, auto-renewing TLS certificate was then issued and installed using Let's Encrypt via Certbot, upgrading the site from HTTP to HTTPS:

```
sudo apt install -y certbot python3-certbot-apache
sudo certbot --apache -d fussy0064.duckdns.org
```

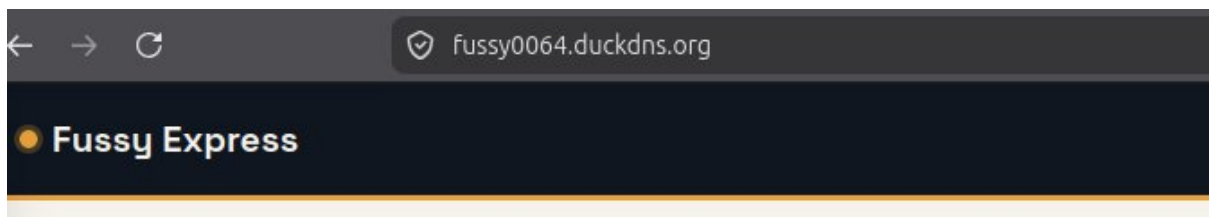
The application itself required no code changes to support HTTPS: session cookies are marked secure automatically once the app detects an HTTPS request (checking both the HTTPS server variable and the X-Forwarded-Proto header), and all internal links are generated relative to the current request scheme.

5.9 File Permissions and Firewall Hardening

```
sudo chown -R www-data:www-data /var/www/TicketBooking-System
sudo find /var/www/TicketBooking-System -type d -exec chmod 755 {} \;
sudo find /var/www/TicketBooking-System -type f -exec chmod 644 {} \;
sudo ufw allow 22,80,443/tcp && sudo ufw enable
```

5.10 Verifying the Live Deployment

With the virtual host active and DNS pointing at the instance, the application was verified by visiting <https://fussy0064.duckdns.org/> in a browser and confirming the search page loads, then logging in as Super Admin/Admin and confirming the dashboard, booking flow, and reports all function against the live MySQL database.

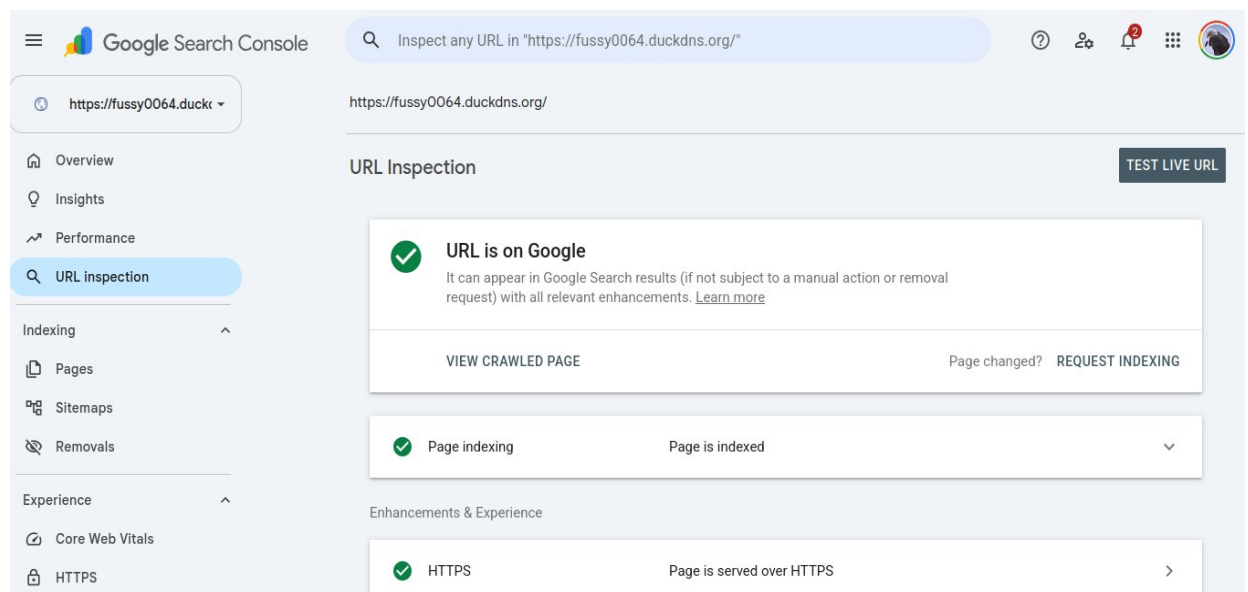


Live site home page at <https://fussy0064.duckdns.org/>

CHAPTER SEVEN: CHALLENGES ENCOUNTERED

- Balancing encryption with searchability: because sensitive columns are encrypted, ordinary SQL WHERE clauses cannot filter on them directly. This was resolved by storing an additional one-way SHA-256 hash of the normalized plain text alongside each encrypted column, purely for lookups and uniqueness checks.
- Preventing double-booked seats under concurrency: an early version only checked the total available-seat count, which could theoretically let two passengers claim the same seat number. This was resolved by locking the schedule row for the duration of the booking transaction and explicitly re-checking the requested seat against all pending/confirmed bookings before saving.
- Avoiding accidental data loss from cascading deletes: the database's foreign-key ON DELETE CASCADE rules meant deleting a bus or route could silently wipe historical bookings and revenue records. Guard checks were added so the application refuses such a delete whenever booking history would be lost.
- Keeping the UI usable on mobile: tables with many columns (e.g., the Users and Bookings pages) needed a responsive layout and a compact actions menu, which required careful CSS so drop-down menus were not clipped by the table's own layout.
- Configuring HTTPS on a free dynamic domain: obtaining a trusted TLS certificate required first pointing a DuckDNS subdomain at the EC2 instance's public IP, then issuing the certificate through Certbot against that domain name rather than the raw IP address.

GOOGLE CONSOLE



CHAPTER EIGHT: RECOMMENDATIONS AND FUTURE WORK

- Integrate a real payment gateway (e.g., mobile money APIs common in Tanzania) so fares can be collected and reconciled automatically at the point of booking.
- Add SMS notifications alongside email, since not all passengers check email regularly.
- Move the encryption key from a configuration file to a managed secret store such as AWS Secrets Manager or SSM Parameter Store for production use.
- Add automated unit tests (e.g., with PHP Unit) around the booking/seat-locking logic to guard against future regressions.
- Introduce a proper CI/CD pipeline so future changes are automatically tested and deployed to the EC2 instance rather than deployed manually via git pull.
- Allow an Elastic IP and a paid, permanent domain name once the project moves beyond a student deployment, since a Free Tier EC2 instance's IP can change on stop/start.

CONCLUSION

Fussy Express successfully demonstrates a complete, secure, object-oriented bus ticket booking system built in plain PHP and MySQL without the use of any framework. The system fulfils every functional requirement identified in Chapter Two: passengers can search, view live seat availability, and book tickets; administrators can manage the fleet and review/confirm bookings; and a Super Admin can securely manage user accounts. The design demonstrates all four pillars of Object-Oriented Programming, a properly normalized (3NF) database, application-level AES-256 encryption of sensitive data, bcrypt password hashing, and a range of additional security controls (CSRF protection, prepared statements, output escaping, role-based access control). The completed application has been deployed and is publicly reachable over HTTPS on an AWS EC2 instance at <https://fussy0064.duckdns.org/>, satisfying the hosting requirement of the assignment.

REFERENCES

- PHP Documentation. (n.d.). PHP: Hypertext Preprocessor Manual. Retrieved from <https://www.php.net/docs.php>
- MySQL Documentation. (n.d.). MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc/>
- OpenSSL Project. (n.d.). openssl_encrypt / openssl_decrypt. PHP Manual. Retrieved from https://www.php.net/openssl_encrypt
- Amazon Web Services. (n.d.). Amazon EC2 Documentation. Retrieved from <https://docs.aws.amazon.com/ec2/>
- DuckDNS. (n.d.). Free Dynamic DNS Service. Retrieved from <https://www.duckdns.org/>
- Electronic Frontier Foundation / Let's Encrypt. (n.d.). Certbot Documentation. Retrieved from <https://certbot.eff.org/>
- OWASP Foundation. (n.d.). OWASP Top Ten and Cheat Sheet Series (CSRF, SQL Injection, XSS). Retrieved from <https://owasp.org/>